

1 Architecture

Figure 1 shows the organisation of the accelerator. The design partitions attention at a single point. Every operation whose cost is constant in context length remains on the host, while the KV cache and the scan over it are placed in programmable logic. The block is invoked once per layer and once per decode step. A single invocation processes that layer’s H KV heads one after another, and within each head the G query heads that share it proceed in parallel. In the model evaluated here $H=8$ and $G=4$, giving 32 attention heads of dimension $d=64$.

1.1 Partitioning

The host computes the token embedding, the three projections W_q , W_k and W_v , and applies rotary position embedding. Position therefore enters the datapath before the cache and never within it.

The target platform is a Xilinx ZCU104, whose Zynq UltraScale+ device couples a processing system to programmable logic through AXI4 interfaces. Vectors cross to the accelerator over a 512-bit AXI4-Stream port, each of the d lanes carried as a 24-bit fixed-point value. Every invocation receives H groups of $2+G$ vectors, namely one key, one value and G queries for each KV head.

Keys and values are encoded and written to DDR before the scan begins, because a token attends to itself as well as to every token before it. The scan therefore covers T rows, where T counts the tokens seen so far including the current one.

1.2 Rotation and quantisation

The rotation unit `rot_fwht` applies one round of a randomised Walsh–Hadamard rotation to keys, values and queries alike. All subsequent arithmetic is performed in this rotated basis, and the inverse rotation is folded into W_o offline so that none is performed at run time.

The norm unit `rot_norm` computes the root-mean-square of the rotated vector at the precision at which it will be stored. It is the only producer of this norm, which guarantees that the quantiser’s decision thresholds and the eventual decoder operate on the same value.

The encoder `rot_encode` maps each rotated vector to codewords without division. Comparing a normalised channel against a threshold is equivalent to comparing the raw channel against that threshold scaled by the norm, so the thresholds are scaled once per vector and each of the d channels reduces to a comparison. Separate instances serve keys and values, which differ in codeword width and in threshold table.

1.3 The cache row

For one token and one KV head the cache holds d key codewords, d value codewords, and two scalar norms, one for the key vector and one for the value vector. Writing

b_k and b_v for the codeword widths and w_n for the norm width, a row occupies

$$R = \underbrace{db_k}_{\text{key codewords}} + \underbrace{db_v}_{\text{value codewords}} + \underbrace{2w_n}_{\text{two norms}} \text{ bits.} \quad (1)$$

In the configuration built, $d=64$, $b_k=4$, $b_v=2$ and $w_n=16$, so a row is 416 bits, or 52 bytes. The write unit `kv.write` assembles this row and issues it to DDR.

1.4 Cache layout

The cache resides entirely in DDR and is stored transposed. A row is partitioned into P slices, and each slice is held as a contiguous array spanning all tokens.

This layout follows from a bandwidth constraint. The scan consumes one row per cycle, while a single 128-bit AXI high-performance port supplies 16 bytes per cycle. P ports must therefore operate concurrently, and each requires its own sequential access stream. A row-major layout contains only one such stream. Partitioning the token range across ports does not help either, because the causal scan advances through tokens in order and would leave all but one port idle. Under the transposed layout each port traverses a single slice, and the P streams jointly deliver one complete row every cycle.

A slice never spans a field boundary, so the port requirement is not a function of row size alone. The key codewords, the value codewords and the norms must together occupy at most four slices, which is the number of high-performance ports the device provides. At 250 MHz the scan sustains 1.14 cycles per row against a one-cycle ideal.

1.5 Scan datapath

Attention is computed directly on codewords rather than on reconstructed vectors. A cached key is the product of its norm and a codebook entry selected by its codeword, so the inner product with a query is a sum over channels of query element times codebook entry, scaled once by that norm. The per-channel product can therefore take only 2^{b_k} distinct values, one for each codebook entry.

Those 2^{b_k} products are therefore evaluated for every channel once per query vector and held on chip. Each cached token afterwards costs d table lookups, an adder tree, and a single multiplication by the row norm, which is two DSP slices per lane rather than d .

The read engine `kv_store_ddr` is instantiated once per block and shared by all G lanes, so one row fetch serves G query heads. This makes the reuse already present in grouped-query attention explicit in the hardware.

Each lane then scores, normalises and accumulates. The scoring unit `score_lane` produces the score for the row. The softmax unit `softmax_online` maintains a running maximum and rescales the accumulator whenever that

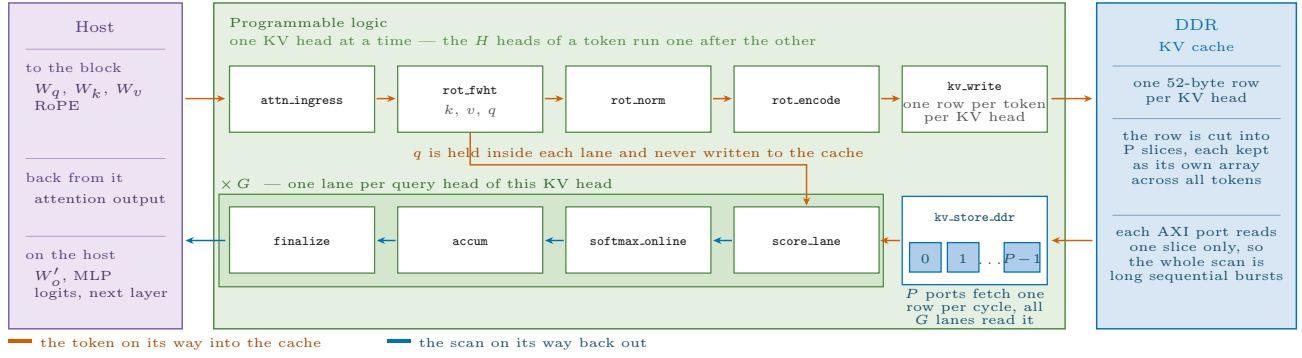


Figure 1: The accelerator. Attention is split once. Everything whose cost is constant in context length stays on the host, while the KV cache and the scan over it sit in programmable logic. Vermilion follows a token on its way into the cache and blue follows the scan on its way back out. The read engine sits outside the replicated lane array because one row fetch serves all G query heads of a KV head. The ZCU104 provides four AXI high-performance ports, each carrying 16 bytes a cycle, and a slice never spans a field boundary, so the key codewords, the value codewords and the norms must together fall into at most four slices.

maximum grows, which allows the scan to finish in a single pass. A two-pass formulation would require a second traversal of an off-die cache and is not affordable. The exponential it needs is evaluated by splitting the argument into a shift and an index into a small table, so that table need only cover the unit interval however large the argument becomes. The accumulator **accum** weights the decoded value and adds it to a running sum, exploiting the same property on the value side, since with 2^{b_v} value centroids one multiplication produces a shared factor and every channel reduces to a multiplexer and an adder. The normalisation unit **finalize** inverts the softmax denominator once per head and multiplies across channels, the denominator not being final until the scan ends.

1.6 Egress

Each group returns its G outputs over the same 512-bit stream, after which the block advances to the next KV head. The host then applies W_o , followed by the MLP and the residual connection. A decode step completes in 22 to 26 microseconds on the device.

Prompt tokens traverse this path identically, so every row the scan reads was written by the accelerator itself.